ServeEase Database Design

Normalization

1NF (First Normal Form)

Normalization to **1NF** reduced repeating attributes and multi-valued fields in the relations.

● Repeating or multi-valued information was separated into independent relations instead of storing multiple values inside a single tuple.
● Multiple customer addresses were separated through the **CUSTOMER_ADDRESS** relation instead of storing many addresses inside **CUSTOMER**.
● Multiple provider working locations were separated through the **PROVIDER_SERVICE_LOCATION** relation instead of storing address lists inside **SERVICE_PROVIDER**.
● Booking updates were separated into **BOOKING_MODIFICATION** instead of storing multiple modification records inside **BOOKING**.
● Atomic attributes were ensured in relations such as **CUSTOMER**, **SERVICE_PROVIDER**, **SERVICE_LISTING**, **BOOKING**, and **PAYMENT**, so fields like email, phone, rating, price, and status contain single values only.
● This reduced duplication within rows and improved clarity of relations such as **BOOKING**, **SERVICE_REQUEST**, **SUPPORT_TICKET**, and **PROVIDER_DOCUMENT**.

2NF (Second Normal Form)

Normalization to **2NF** reduced partial dependencies on composite or relationship-based data.

● Address details were separated from customer and provider data:
○ **ADDRESS(address_id, address_line, city, state, pincode, country)**
○ Linked through **CUSTOMER_ADDRESS(customer_id, address_id)** and **PROVIDER_SERVICE_LOCATION(provider_id, address_id)**

● Service category information was separated from service listings:
○ **SERVICE_CATEGORY(category_id, category_name, description)**
○ **SERVICE_LISTING(service_id, provider_id, category_id, service_name, ...)**

● Booking-related transactional details were separated into specialized relations:
○ **PAYMENT(payment_id, booking_id, amount, payment_method, payment_status)**

○ **CANCELLATION(cancellation_id, booking_id, cancelled_by, refund_status)**
○ **BOOKING_MODIFICATION(modification_id, booking_id, old_slot_id, new_slot_id, ...)**

● Provider document verification details were separated from provider account details:
○ **SERVICE_PROVIDER(provider_id, name, email, phone, ...)**
○ **PROVIDER_DOCUMENT(document_id, provider_id, document_type, verification_status, ...)**

● This reduced duplicate storage of address, service category, payment, cancellation, and document data across multiple records.

3NF (Third Normal Form)

Normalization to **3NF** reduced transitive dependencies between non-key attributes.

● Address information was separated from customer, booking, and provider service location data:
○ **ADDRESS(address_id, address_line, city, state, pincode, country)**
○ Linked to **SERVICE_REQUEST**, **BOOKING**, **CUSTOMER_ADDRESS**, and **PROVIDER_SERVICE_LOCATION** through foreign keys.

● Role-specific user data was separated into different relations:
○ **CUSTOMER**
○ **SERVICE_PROVIDER**
○ **ADMIN**
○ **CUSTOMER_SUPPORT_STAFF**

● Payment details were separated from booking details:
○ **PAYMENT(payment_id, booking_id, amount, payment_status, transaction_id)**

● Review information was separated from provider and booking records:
○ **REVIEW_RATING(review_id, booking_id, customer_id, provider_id, rating, review_text)**

● Support issue information was separated from booking records:
○ **SUPPORT_TICKET(ticket_id, customer_id, booking_id, support_id, issue_type, description, ticket_status)**

● Notification data was separated from customer, provider, and admin core records:
○ **NOTIFICATION(notification_id, user_id, user_role, message, notification_type, status)**

● This reduced indirect attribute dependencies and redundant storage of address, payment, review, support, and notification information.

BCNF (Boyce–Codd Normal Form)

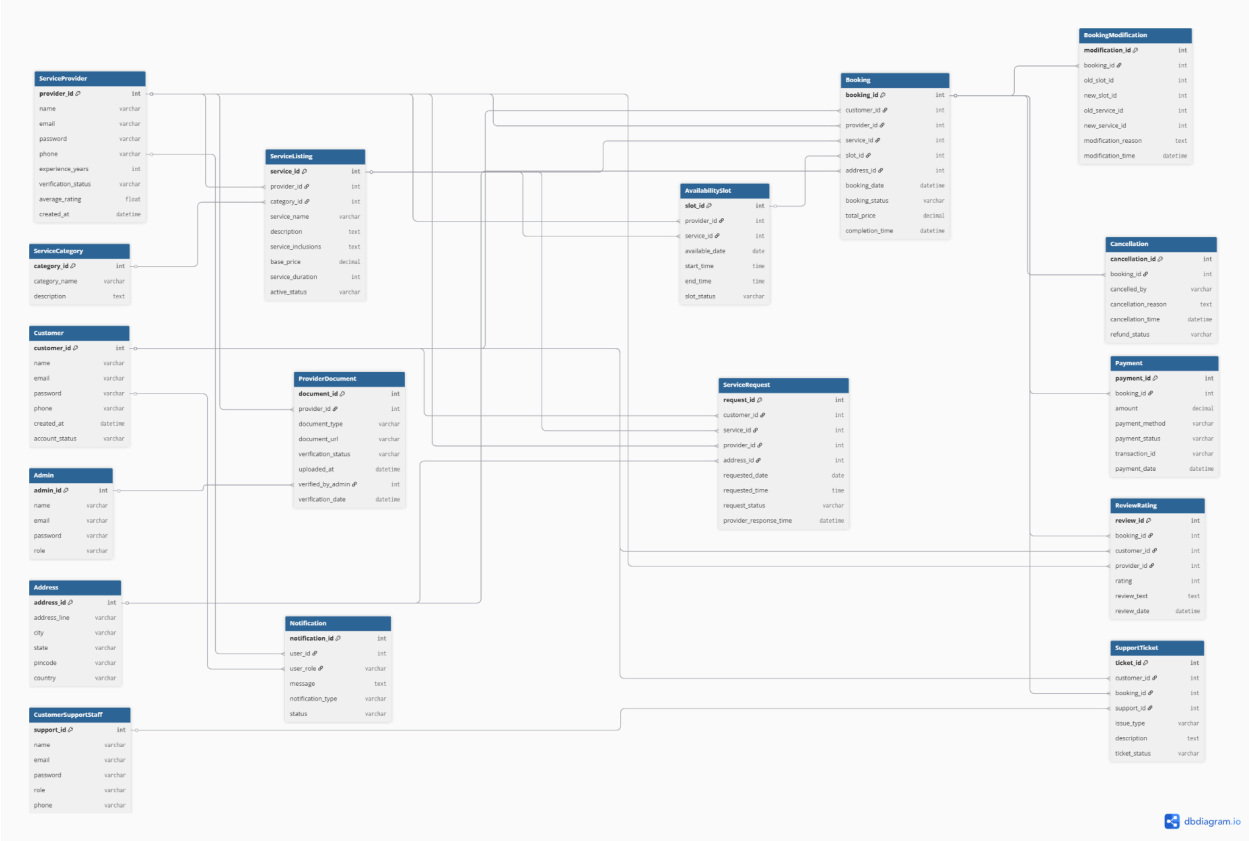
Normalization to **BCNF** ensured that every relation is fully dependent on its primary key, further reducing anomalies.

- Each entity now has a clear primary key dependency:
  - **CUSTOMER → customer_id**
  - **SERVICE_PROVIDER → provider_id**
  - **SERVICE_CATEGORY → category_id**
  - **SERVICE_LISTING → service_id**
  - **SERVICE_REQUEST → request_id**
  - **BOOKING → booking_id**
  - **PAYMENT → payment_id**
  - **REVIEW_RATING → review_id**
  - **SUPPORT_TICKET → ticket_id**
  - **PROVIDER_DOCUMENT → document_id**

- Determinants such as **booking_id**, **service_id**, **provider_id**, **request_id**, and **document_id** uniquely determine all attributes in their respective relations.

- Relations like **AVAILABILITY_SLOT**, **PAYMENT**, **CANCELLATION**, **BOOKING_MODIFICATION**, and **PROVIDER_DOCUMENT** store only attributes directly related to their primary keys.

- Associative relations such as **CUSTOMER_ADDRESS** and **PROVIDER_SERVICE_LOCATION** resolve many-to-many relationships without introducing redundancy.

- This reduced **update**, **insertion**, and **deletion anomalies** across the schema.

Relational Schema: